

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Процессы и профессиональная практика программной инженерии»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Алешов Е.В.

ПРАКТИЧЕСКАЯ РАБОТА 4

«Архитектура и функциональные возможности TFS»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью практической работы является изучение архитектуры и функциональных возможностей Microsoft Visual Studio Team Foundation Server 2012, анализ вариантов его развёртывания, сравнение шаблонов командных проектов и разработка структуры групп безопасности для организации совместной работы команды разработчиков программного обеспечения.

1.2. Задачи

- Изучить трёхуровневую архитектуру Team Foundation Server 2012.
- Определить назначение клиентского уровня, уровня приложений и уровня данных.
- Объяснить, в каких случаях целесообразно расширять функциональность TFS с помощью серверной объектной модели, и привести пример расширения.
- Сравнить простейшую, простую и среднюю серверные топологии TFS.
- Выбрать подходящую топологию для стартапа, компании среднего размера и крупной корпорации.
- Сравнить шаблоны MSF for CMMI Process Improvement 6.0, MSF for Agile Software Development 6.0 и Microsoft Visual Studio Scrum 2.2.
- Разработать структуру групп безопасности для команды проекта.
- Определить права чтения, записи, создания веток и управления сборками для каждой группы.
- Обосновать предложенные решения с учётом производительности, масштабируемости и безопасности.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Назначение и функциональные возможности TFS

Microsoft Visual Studio Team Foundation Server 2012 представляет собой серверную платформу для организации совместной разработки программного обеспечения. Она обеспечивает централизованное управление проектами, рабочими элементами, исходным кодом, тестированием, сборками и отчётностью. Использование TFS позволяет согласовывать действия участников команды, контролировать выполнение задач и сохранять историю изменений программного продукта.

2.2. Трёхуровневая архитектура TFS

Архитектура TFS 2012 является трёхуровневой и включает клиентский уровень, уровень приложений и уровень данных. Клиентский уровень обеспечивает взаимодействие пользователей с системой. Уровень приложений содержит серверные службы, реализующие бизнес-логику. Уровень данных отвечает за хранение конфигурации, рабочих элементов, исходного кода и аналитической информации. Разделение уровней позволяет разграничивать обязанности компонентов и выбирать подходящую конфигурацию серверной инфраструктуры.

2.3. Клиентский уровень

Клиентский уровень обеспечивает доступ пользователей к функциям TFS. Для работы могут использоваться Visual Studio, веб-браузер, Microsoft Excel, Microsoft Project, командные инструменты и сторонние средства интеграции. С помощью этих инструментов разработчики получают доступ к рабочим элементам, репозиториям, сборкам, тестам и другим ресурсам командного проекта. Клиентские приложения взаимодействуют с серверными компонентами через предусмотренные программные интерфейсы и веб-службы.

2.4. Уровень приложений

Уровень приложений реализует основные функции TFS и в рассматриваемой архитектуре TFS 2012 размещается в среде IIS с использованием веб-технологий ASP.NET. Он включает службы управления версиями, рабочими элементами, сборками, тестированием и другие платформенные компоненты. Серверная объектная модель предоставляет программный интерфейс для взаимодействия с серверной функциональностью. Она позволяет создавать расширения, автоматизировать операции и интегрировать TFS с корпоративными системами.

2.5. Уровень данных

Уровень данных основан на Microsoft SQL Server 2012 и включает несколько баз данных и аналитическое хранилище. В зависимости от конфигурации используются база данных конфигурации сервера, базы данных коллекций командных проектов, аналитические компоненты и хранилище данных. Операционная информация сохраняется в базах данных, а сведения для аналитики и формирования отчётов передаются в соответствующие хранилища по установленному расписанию. Разделение серверных ролей позволяет выделять ресурсы для обработки запросов и хранения информации.

2.6. Расширение функциональности через серверную объектную модель

Серверную объектную модель целесообразно использовать, если необходимо программно расширить возможности TFS, автоматизировать сложные операции или реализовать собственную бизнес-логику на стороне сервера. Веб-службы предоставляют предусмотренные системой операции, тогда как объектная модель позволяет разработчику обращаться к серверным функциям через программные интерфейсы и строить на их основе дополнительные компоненты. При выборе способа расширения необходимо учитывать поддерживаемые API, права доступа и совместимость с конкретной версией TFS.

2.7. Основные функциональные возможности TFS

- Управление проектами - организация командных проектов, планирование работ и контроль выполнения задач.
- Отслеживание рабочих элементов - регистрация требований, задач, ошибок, тестовых случаев и препятствий, а также контроль их состояний и истории.
- Контроль версий - хранение исходного кода, ведение истории изменений, создание веток и объединение изменений.
- Управление тестовыми случаями - организация тестирования, хранение тестовых сценариев и отслеживание результатов проверок.
- Автоматизация построения - настройка сборок программного продукта вручную, по расписанию или при изменении исходного кода.
- Отчётность - получение сведений о ходе проекта, состоянии рабочих элементов, результатах сборок и других показателях разработки.

2.8. Варианты развёртывания TFS

TFS может развёртываться на одном сервере или на нескольких серверах с разделением ролей. Простейшая топология предполагает размещение основных компонентов на одном физическом сервере и подходит для небольших команд с ограниченной нагрузкой. Простая топология предусматривает выделение ресурсов для построения и тестирования на дополнительных компьютерах. Топология средней сложности разделяет уровень приложений и уровень данных, а также позволяет выделять отдельные серверы для SharePoint, служб отчётности, сборок и тестирования.

2.9. Шаблоны командных проектов

Шаблон командного проекта определяет структуру рабочих элементов, правила организации процесса разработки, классификации, отчёты, разрешения и другие настройки проекта. В TFS 2012 рассматриваются шаблоны MSF for CMMI Process Improvement 6.0, MSF for Agile Software Development 6.0 и Microsoft Visual Studio Scrum 2.2. Они предназначены для различных подходов к управлению разработкой: формализованного процессного управления, гибкой разработки и работы по Scrum.

2.10. Безопасность и разграничение доступа

Безопасность TFS обеспечивается посредством групп пользователей и разрешений. Права определяют, какие действия участники могут выполнять с исходным кодом, рабочими элементами, ветками и сборками. Для упрощения администрирования пользователей объединяют по ролям: руководители проекта, архитекторы, разработчики, тестировщики и администраторы. Принцип минимальных привилегий предполагает предоставление только тех прав, которые необходимы для выполнения должностных обязанностей.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Анализ архитектуры

- Рассматривается взаимодействие трёх уровней TFS 2012.
- Определяется назначение клиентских приложений и серверных служб.
- Анализируются функции серверной объектной модели и веб-служб.
- Устанавливается связь между серверными службами и базами данных.
- Формулируется пример программного расширения TFS.

3.2. Выбор серверной топологии

- Для каждой компании учитываются численность команды, частота сборок, требования к производительности и отчётности.
- Для стартапа рассматривается простейшая односерверная конфигурация.
- Для компании среднего размера выбирается простая топология с выделенными ресурсами построения и тестирования.
- Для крупной корпорации выбирается топология средней сложности с разделением уровней приложений и данных, а также выделением серверов для дополнительных служб.

3.3. Сравнение шаблонов процессов

Шаблоны сравниваются по целевой аудитории, уровню формализации, особенностям планирования и типичным сценариям применения. Для СММІ рассматривается формализованное управление процессами, для Agile - гибкое управление требованиями и итерациями, для Scrum - организация работы через спринты, элементы бэклога и роли Scrum.

3.4. Планирование безопасности

- Создаются логические группы по ролям участников проекта.
- Для каждой группы определяются права чтения и изменения данных.
- Отдельно рассматриваются создание и изменение веток исходного кода.
- Управление сборками предоставляется только участникам, которым оно необходимо.
- Административные права отделяются от повседневной разработки.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Задание 1. Анализ архитектуры TFS

Расширение функциональности через серверную объектную модель требуется в тех случаях, когда стандартных возможностей TFS недостаточно для реализации специфических процессов компании. Такой подход подходит для создания собственного серверного компонента, автоматизации работы с рабочими элементами и интеграции TFS с внутренними информационными системами. Веб-службы удобны для обращения к уже реализованным операциям, а серверная объектная модель используется как программная основа для разработки расширений с учётом возможностей конкретной версии платформы.

4.1.1. Пример расширения функциональности

Примером является создание серверного расширения для автоматической проверки рабочих элементов перед передачей задачи на тестирование. После изменения состояния задачи расширение проверяет заполнение обязательных полей, наличие связанного требования и привязку изменений исходного кода. Если необходимые условия не выполнены, система формирует уведомление или не позволяет автоматизированному процессу продолжить обработку задачи. Реализация должна использовать поддерживаемые механизмы расширения TFS 2012 и проверять права пользователя, выполняющего операцию.

4.1.2. Обоснование выбора серверной объектной модели

Серверная объектная модель позволяет строить расширения на основе программных интерфейсов TFS и взаимодействовать с объектами платформы без необходимости самостоятельно реализовывать все базовые механизмы управления проектом. Это снижает дублирование логики и упрощает интеграцию с существующими процессами. При этом серверная объектная модель не означает возможность произвольно изменять внутренние базы данных или обходить стандартные механизмы безопасности. При разработке необходимо учитывать ограничения API, документацию и совместимость компонентов.

4.2. Задание 2. Выбор топологии для трёх компаний

Выбор топологии определяется нагрузкой на систему, количеством пользователей, частотой построений и требованиями к хранению данных. Односерверная конфигурация снижает затраты на инфраструктуру, но ограничивает возможности масштабирования. Выделение серверов построения уменьшает конкуренцию за вычислительные ресурсы. Разделение уровней приложений и данных позволяет гибче управлять нагрузкой и обслуживать крупные команды.

4.2.1. Компания 1 - стартап из 15 разработчиков

- Рекомендуемая топология: простейшая односерверная.
- Обоснование: команда небольшая, бюджет ограничен, а сборки выполняются редко. Размещение основных компонентов на одном сервере позволяет сократить расходы на оборудование и администрирование.
- Состав: компоненты TFS и SQL Server размещаются на одном физическом сервере в соответствии с требованиями установки выбранной версии.
- Построения и тестирование: при небольшой нагрузке могут выполняться на клиентских компьютерах или на том же сервере, если это допустимо по ресурсам и требованиям безопасности.
- Ограничения: необходимо контролировать потребление процессорного времени, памяти и дискового пространства. При увеличении нагрузки следует рассмотреть выделение серверов построения.

4.2.2. Компания 2 - 80 разработчиков

- Рекомендуемая топология: простая серверная.
- Обоснование: ежедневное выполнение большого количества сборок создаёт дополнительную нагрузку на процессор, оперативную память и дисковую подсистему. Выделение построения и тестирования на отдельные компьютеры снижает конкуренцию за ресурсы основного сервера.
- Состав: веб-службы TFS и базы данных размещаются на основном сервере, а контроллеры построения и тестирования - на дополнительных компьютерах.
- Преимущества: уменьшение влияния сборок на интерактивную работу пользователей, более эффективное распределение ресурсов и возможность независимого масштабирования вычислительной нагрузки.
- Ограничения: если база данных становится узким местом, следует оценить возможность дальнейшего разделения уровней приложений и данных.

4.2.3. Компания 3 - крупная корпорация с 500 и более разработчиками

- Рекомендуемая топология: средняя серверная.
- Обоснование: большое количество пользователей, несколько команд и продуктов, интенсивная отчётность и использование SharePoint требуют распределения нагрузки между отдельными серверными ролями.
- Состав: уровень приложений TFS размещается на отдельном сервере, уровень данных SQL Server - на другом. Для SharePoint и служб отчётности выделяются отдельные серверы или соответствующие инфраструктурные ресурсы. Серверы построения и контроллеры тестирования также могут размещаться отдельно.
- Преимущества: улучшение масштабируемости, возможность выделения ресурсов для наиболее нагруженных служб и более гибкое сопровождение корпоративной инфраструктуры.
- Дополнительные меры: следует предусмотреть резервное копирование баз данных, мониторинг, контроль доступа, план восстановления после сбоев и анализ производительности.
- Ограничения: средняя топология увеличивает затраты на инфраструктуру и администрирование. При высоких требованиях к доступности и нагрузке может потребоваться более сложная конфигурация.

4.2.4. Сводная таблица выбора топологии

Компания	Число разработчиков	Топология	Основное обоснование
Стартап	1-5	Простейшая	Минимальные затраты и небольшая нагрузка
Компания среднего размера	80	Простая	Выделение ресурсов для частых сборок и тестирования
Крупная корпорация	500+	Средняя	Разделение серверных ролей, поддержка SharePoint и отчётности

Выбор соответствует условиям учебного задания. В реальной инфраструктуре окончательное решение принимается после оценки нагрузки, требований к доступности и ограничений конкретной версии TFS.

4.3. Задание 3. Сравнение шаблонов процессов TFS 2012

Шаблоны командных проектов отличаются подходами к управлению требованиями, планированию работ, контролю качества и формализации процессов. СММІ ориентирован на управляемые и документируемые процессы, Agile - на гибкое изменение требований и регулярную поставку результата, Scrum - на организацию разработки через спринты и определённые роли и артефакты Scrum.

4.3.1. Таблица сравнения шаблонов

Критерий	MSF for CMMI Process Improvement 6.0	MSF for Agile Software Development 6.0	Microsoft Visual Studio Scrum 2.2	Основной подход
Формализованное управление процессами	Гибкая разработка программного обеспечения	Методология Scrum	Целевая аудитория	Команды с высокими требованиями к регламентации и контролю
Команды, работающие итеративно и адаптирующиеся к изменениям	Команды, использующие роли и артефакты Scrum	Управление требованиями	Подробное описание требований и их прослеживаемость	Пользовательские истории и требования, уточняемые по мере работы
Элементы Product Backlog и требования, уточняемые в ходе разработки	Планирование	Планирование с учётом формализованных процессов и критериев	Итеративное планирование с возможностью изменения приоритетов	Планирование спринтов и управление бэклогом
Рабочие элементы	Требования, задачи, ошибки и элементы контроля процесса	Пользовательские истории, задачи, ошибки и другие элементы Agile	Элементы Product Backlog, задачи, ошибки и элементы Scrum	Контроль качества
Акцент на процессах, проверках и документировании	Регулярная проверка результата и обратная связь	Проверка инкремента и контроль выполнения критериев готовности	Гибкость	Ниже при строгой регламентации, но допускается настройка процессов
Высокая	Высокая в рамках правил Scrum	Типичные области применения	Проекты с повышенными требованиями к прослеживаемости и управляемости	Продуктовая разработка с меняющимися требованиями
Команды, работающие по Scrum и использующие спринты	Преимущество	Структурированность и контроль процесса	Адаптивность и оперативная обратная связь	Понятная организация работы и регулярные итерации
Ограничение	Дополнительные затраты на документирование и соблюдение процедур	Требует регулярной коммуникации и дисциплины команды	Требует соблюдения практик Scrum и согласованности участников	

4.3.2. Обоснование выбора шаблона

Для организации с обязательной документацией, формальными процедурами и повышенными требованиями к прослеживаемости рекомендуется MSF for CMMI Process Improvement 6.0. Для команды, которая часто меняет требования и стремится быстро получать обратную связь от заказчика, подходит MSF for Agile Software Development 6.0. Для команды, которая организует работу по Scrum, использует спринты и ведёт бэклог продукта, целесообразно выбрать Microsoft Visual Studio Scrum 2.2. Окончательный выбор зависит от фактического процесса разработки и требований организации.

4.4. Задание 4. Планирование безопасности

Для проекта предлагается создать отдельные группы безопасности в соответствии с обязанностями участников. Менеджер проекта отвечает за планирование и контроль работ, архитектор определяет структуру решения, разработчики создают и изменяют программный код, тестировщики выполняют проверки, а администратор TFS обслуживает серверную инфраструктуру и управляет разрешениями. Группы позволяют централизованно назначать права и изменять их при изменении состава команды.

4.4.1. Структура групп безопасности

- TFS_Project_Managers - менеджер проекта. Отвечает за планирование работ, контроль сроков, просмотр состояния задач и координацию команды.
- TFS_Architects - архитектор. Отвечает за архитектурные решения, проектирование компонентов, согласование изменений и контроль структуры программного продукта.
- TFS_Developers - три разработчика. Выполняют реализацию функций, исправление ошибок, внесение изменений в исходный код и работу с ветками.
- TFS_Testers - два тестировщика. Создают и выполняют тестовые случаи, регистрируют ошибки и проверяют исправления.
- TFS_Administrators - администратор TFS. Управляет конфигурацией сервера, группами безопасности, разрешениями и инфраструктурой сборки.

4.4.2. Матрица разрешений

Группа	Чтение	Запись	Создание	веток	Управление	сборками
Менеджеры проекта	Да	Да, для рабочих элементов и планов проекта	Нет, по умолчанию	Нет	Нет	Нет
Архитекторы	Да	Да, для архитектурных документов и согласованных изменений	Да	Нет, только просмотр результатов	Да	Да
Разработчики	Да	Да, для исходного кода и рабочих элементов в рамках обязанностей	Да	Да	Ограниченное управление очередью сборки, если требуется	Да
Тестировщики	Да	Да, для тестовых случаев, результатов и ошибок	Нет, по умолчанию	Нет	Нет	Нет
Администраторы TFS	Да	Да, в рамках административных полномочий	Да	Да	Да	Да

Матрица является предлагаемой политикой доступа. В TFS 2012 права на исходный код, рабочие элементы и сборки назначаются через соответствующие области разрешений. Указанные действия необходимо реализовать посредством конкретных разрешений и групп выбранной конфигурации.

4.4.3. Обоснование прав менеджера проекта

Менеджеру проекта предоставляются права чтения и изменения рабочих элементов, планов и сведений о ходе выполнения задач. Это позволяет корректировать приоритеты, назначать исполнителей и контролировать сроки. По умолчанию менеджеру не требуется изменять исходный код или создавать ветки, поскольку эти операции относятся к разработке. Управление сборками также не предоставляется, если менеджер не выполняет соответствующие технические обязанности.

4.4.4. Обоснование прав архитектора

Архитектору необходим доступ на чтение к исходному коду и документации проекта, а также возможность изменять архитектурные документы и связанные рабочие элементы. Право создавать ветки позволяет организовывать отдельную разработку крупных архитектурных изменений. Полное управление сборками не требуется в повседневной работе, поскольку ответственность за инфраструктуру сборок закрепляется за соответствующими техническими специалистами.

4.4.5. Обоснование прав разработчиков

Разработчикам предоставляются права чтения и записи исходного кода, а также возможность создавать ветки для разработки функций и исправления ошибок. Изменения должны фиксироваться в системе контроля версий и связываться с рабочими элементами, если это предусмотрено процессом проекта. Право управлять конфигурацией сборок не следует автоматически предоставлять каждому разработчику. При необходимости можно разрешить запуск существующих сборок, сохранив изменение их конфигурации за ответственными специалистами.

4.4.6. Обоснование прав тестировщиков

Тестировщикам предоставляется доступ на чтение к программному коду и рабочим элементам, а также право создавать и изменять тестовые случаи, фиксировать ошибки и регистрировать результаты проверок. Возможность записи в исходный код по умолчанию не предоставляется, поскольку тестировщики должны проверять продукт, а не изменять его реализацию. Создание веток и администрирование сборок также ограничиваются, за исключением случаев, когда эти операции входят в утверждённый процесс тестирования.

4.4.7. Обоснование прав администратора TFS

Администратору TFS предоставляются необходимые административные разрешения для управления конфигурацией сервера, группами безопасности, доступом пользователей и инфраструктурой сборок. Эти полномочия позволяют поддерживать работоспособность системы и устранять проблемы доступа. Административные права должны предоставляться только уполномоченному сотруднику и использоваться для выполнения служебных задач. По возможности административная учётная запись отделяется от повседневной учётной записи пользователя.

4.4.8. Дополнительные меры безопасности

- Использовать принцип минимальных привилегий: участник получает только права, необходимые для его роли.
- Не предоставлять всем разработчикам административные права на сервер TFS.
- Разграничивать разрешения на исходный код, рабочие элементы, ветки и сборки.
- Периодически проверять состав групп и отзывать доступ у выбывших участников.
- Ограничивать изменение конфигураций сборок и критически важных веток.
- Использовать историю изменений и результаты проверок для анализа действий участников.
- Проверять, что назначенные разрешения не противоречат ограничениям, унаследованным от родительских групп и областей доступа.

4.5. Общая оценка предложенных решений

Анализ показывает, что архитектура TFS позволяет организовать совместную работу участников проекта, централизованно хранить исходный код и рабочие элементы, автоматизировать сборки и контролировать качество программного продукта. Для небольших команд рационально минимизировать инфраструктурные затраты, а для крупных организаций - разделять серверные роли и распределять нагрузку. Выбор шаблона командного проекта должен соответствовать принятой методологии, а разграничение прав - фактическим обязанностям пользователей.

V. ВЫВОДЫ

В ходе выполнения практической работы были рассмотрены архитектура и основные функциональные возможности Microsoft Visual Studio Team Foundation Server 2012. Установлено, что система включает клиентский уровень, уровень приложений и уровень данных. Клиентские приложения взаимодействуют с серверными службами, уровень приложений реализует функции управления проектами, версиями, рабочими элементами, сборками и тестированием, а уровень данных обеспечивает хранение и обработку информации. Серверная объектная модель может использоваться для создания программных расширений и автоматизации специфических процессов компании при условии соблюдения ограничений API и требований безопасности. Для стартапа из 15 разработчиков выбрана простейшая односерверная топология, для компании из 80 разработчиков - простая топология с выделением ресурсов построения и тестирования, для корпорации с 500 и более разработчиками - средняя топология с разделением серверных ролей и поддержкой дополнительных служб. Сравнение шаблонов показало, что MSF for CMMI ориентирован на формализованные процессы, MSF for Agile - на гибкую разработку, а Microsoft Visual Studio Scrum - на организацию работы по Scrum. Для обеспечения безопасности предложены отдельные группы менеджеров проекта, архитекторов, разработчиков, тестировщиков и администраторов TFS. Права чтения, записи, создания веток и управления сборками распределены с учётом обязанностей участников и принципа минимальных привилегий. Таким образом, выполнение работы позволило сформировать представление о выборе архитектуры TFS, организации командных процессов и разграничении доступа в системе совместной разработки программного обеспечения.